

Práctica 10.

Gallardo Valdez Brayan Alexis Granados Rovira José Pablo
Lozano León Diego Márquez Corona Danna Lizette
Paredes González Emiliano Sebastián Tapia Anrubio Irving Axel

26 de mayo de 2026

1. Script de funciones explicado:

1.1. Función: reconoceredad

Primer La primera función se trata de saber la edad de nuestros clientes a partir de recibir su ID. Recordemos que en nuestro modelo Entidad-Relación la edad era un atributo calculado por lo que en este momento debemos de solucionar ese problema. Para esta función simplemente bastó con recopilar la fecha de nacimiento de nuestro cliente en cuestión y checar que el id del cliente que nos pasaban coincidía con el que estaba en nuestra base de datos. Luego, simplemente con la función DATE_Part obtuvimos el año que era lo que nos interesaba y con la función AGE simplemente calculamos la edad y la regresamos. Adjuntamos imagen de que sí funcionó:

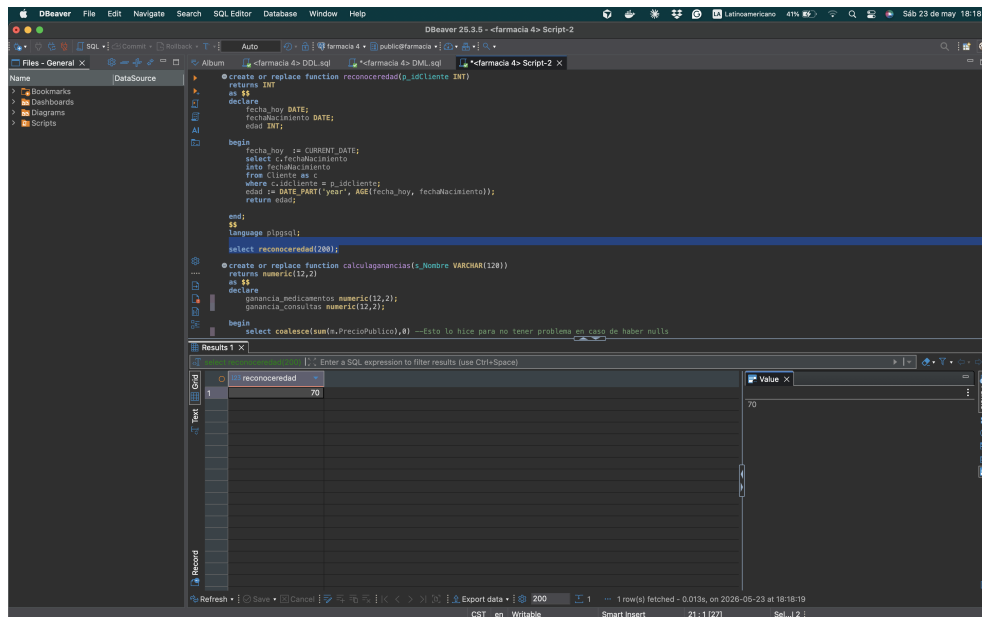
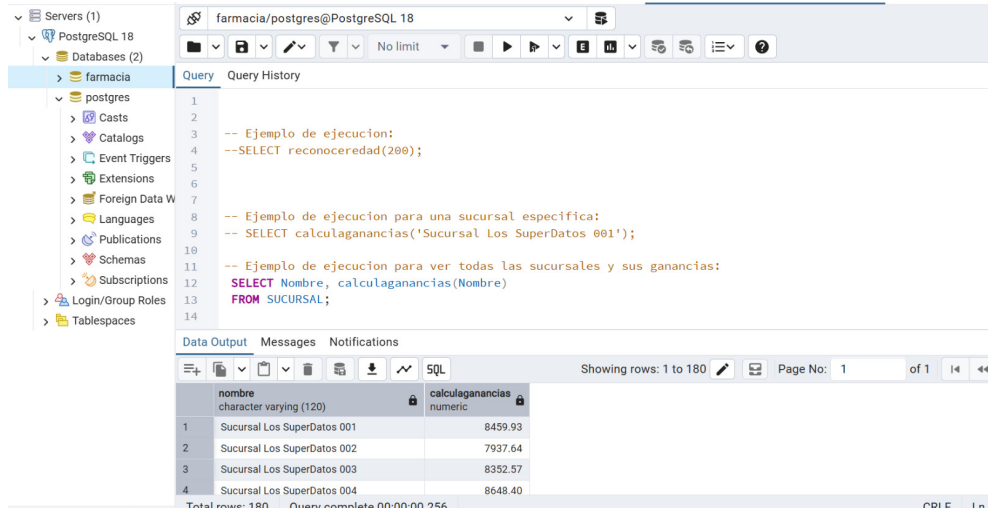


Figura 1: Prueba de que la función reconoceredad funciona.

1.2. Función: calculaganancias

Para la segunda función recibimos el nombre de la sucursal y regresamos un valor numeric pues queremos que tenga decimales. Luego entonces, por medio de coalesce mitigamos el riesgo de tener nulls en nuestra función tal que si encuentra uno le ponga cero pues coalesce devuelve siempre el primer

valor no nulo que se encuentra. Luego bastó con unir las tablas para primero encontrar el costo de los medicamentos en los tickets. Luego unimos igualmente hasta encontrar las ganancias de consultas y al final sumamos ambas y filtramos con un AND el año que queríamos.



Query:

```

-- Ejemplo de ejecución:
--SELECT reconoceredad(200);

-- Ejemplo de ejecución para una sucursal específica:
-- SELECT calculaganancias('Sucursal Los SuperDatos 001');

-- Ejemplo de ejecución para ver todas las sucursales y sus ganancias:
SELECT Nombre, calculaganancias(Nombre)
FROM SUCURSAL;

```

Data Output:

	nombre character varying (120)	calculaganancias numeric
1	Sucursal Los SuperDatos 001	8459.93
2	Sucursal Los SuperDatos 002	7937.64
3	Sucursal Los SuperDatos 003	8352.57
4	Sucursal Los SuperDatos 004	8648.40

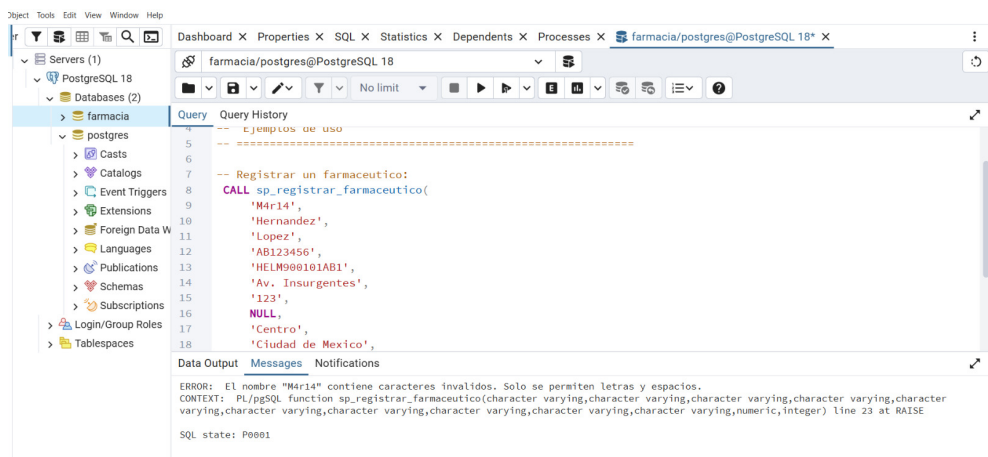
Total rows: 180 Query complete 00:00:00 256

Figura 2: Prueba de que la función calculaganancias funciona.

2. Script de procedimientos almacenados explicado

2.1. Procedimiento: sp_registrar_farmaceutico

Para este procedimiento recibimos los datos de un farmacéutico y realizamos distintas validaciones como verificar que el nombre no venga vacío, que no contenga caracteres inválidos, que la cédula y RFC tengan el tamaño correcto y que el salario no sea negativo. Luego verificamos que la sucursal exista y utilizamos un LOCK para evitar problemas de concurrencia. Después calculamos el siguiente ID disponible e insertamos primero en PERSONAL y luego en FARMACEUTICO.



Query:

```

-- Ejemplos de uso
--
-- Registrar un farmaceutico:
CALL sp_registrar_farmaceutico(
  'M4r14',
  'Hernandez',
  'Lopez',
  'AB123456',
  'HELM900101AB1',
  'Av. Insurgentes',
  '123',
  NULL,
  'Centro',
  'Ciudad de Mexico'
);

```

Data Output:

```

ERROR: El nombre "M4r14" contiene caracteres invalidos. Solo se permiten letras y espacios.
CONTEXT: PL/pgSQL function sp_registrar_farmaceutico(character varying,character varying,character varying,character varying,character varying,character varying,character varying,character varying,character varying,numeric,integer) line 23 at RAISE
SQL state: P0001

```

Figura 3: Prueba de que el procedimiento sp_registrar_farmaceutico funciona.

2.2. Procedimiento: sp_eliminar_medicamento

En este procedimiento recibimos el ID de un medicamento y primero verificamos que exista. Después eliminamos todos los registros relacionados en tablas dependientes para no violar las llaves foráneas.

Finalmente eliminamos el medicamento principal y mostramos cuántas filas fueron afectadas utilizando GET DIAGNOSTICS y RAISE NOTICE.

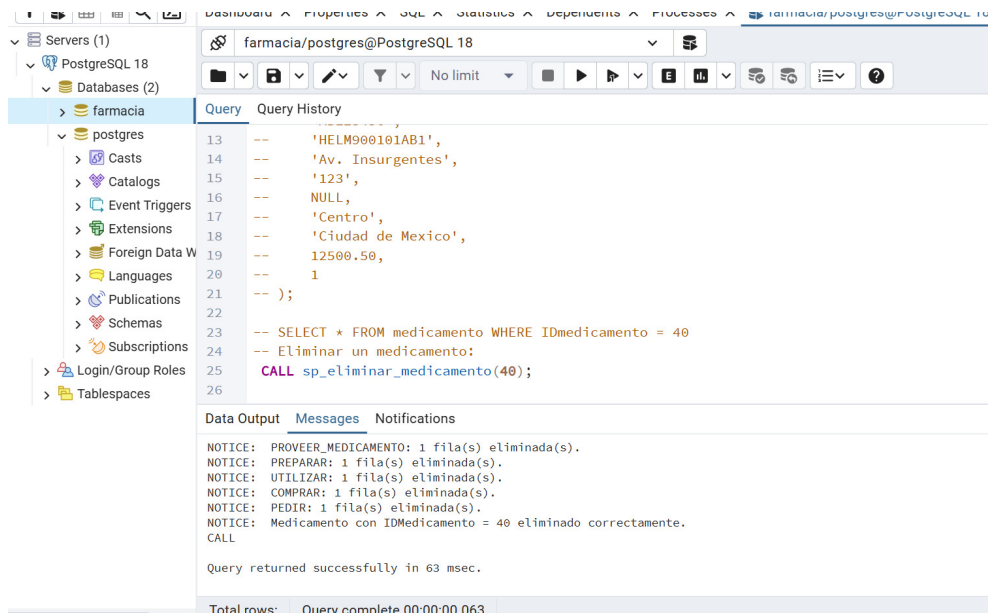
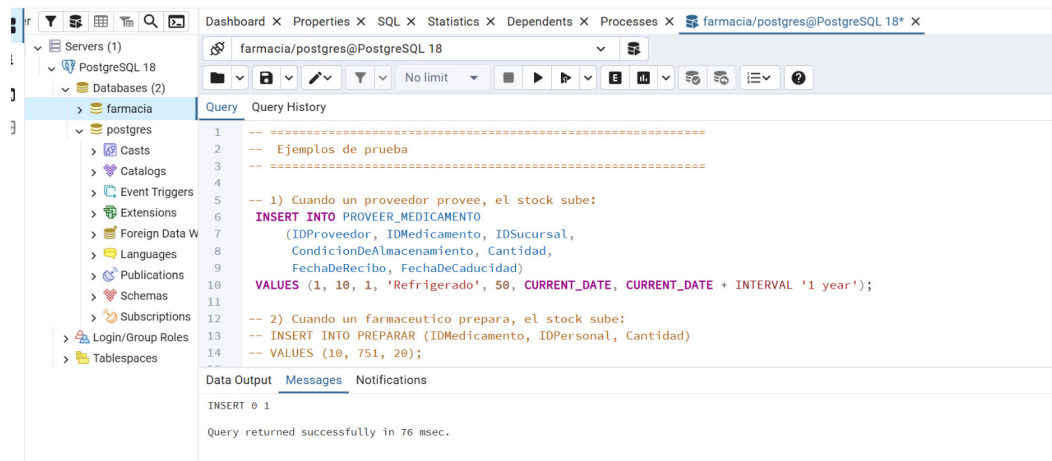


Figura 4: Prueba de que el procedimiento sp_eliminar_medicamento funciona.

3. Script de triggers explicado

3.1. Trigger: trg_stock_proveer_medicamento

Este trigger actualiza automáticamente el stock de medicamentos cuando se inserta, actualiza o elimina un registro en PROVEER_MEDICAMENTO. Si se inserta suma unidades al stock, si se elimina las resta y si se actualiza primero resta la cantidad anterior y luego suma la nueva.



3.2. Trigger: trg_stock_preparar

Este trigger funciona de manera similar al anterior pero sobre la tabla PREPARAR. Su objetivo es mantener actualizado el stock dependiendo de las cantidades de medicamentos preparados.

9	-- FechaDeRecibo, FechaDeCaducidad)
10	-- VALUES (1, 10, 1, 'Refrigerado', 50, CURRENT_DATE, CURRENT_DATE + INTERVAL '1 year');
11	SELECT * FROM medicamento WHERE IDmedicamento = 10
12	-- 2) Cuando un farmaceutico prepara, el stock sube:
13	-- INSERT INTO PREPARAR (IDMedicamento, IDPersonal, Cantidad)
14	-- VALUES (10, 751, 20);
15	
16	-- 3) Cuando un cliente compra, el stock baja y el ticket recalcula precios:
17	-- INSERT INTO TICKET (IDTicket, IDSucursal, IDCliente, Fecha, Hora)
18	-- VALUES (1001, 1, 5, DATE '2026-05-24', TIME '10:00');

acciones	formulacion text	preparadosoficiales boolean	pediatrica boolean	dermatologica boolean	stock integer
1	que erat eros, viverra eget, congue eget, semper rutrum, n...	Donec ut mauris eget massa tempor conval...	true	true	false

3.3. Trigger: trg_stock_comprar

Este trigger se ejecuta cuando un cliente compra medicamentos. Primero verifica que el medicamento exista y que haya suficiente stock disponible. Si no hay inventario suficiente lanza una excepci3n. En caso contrario descuenta las unidades correspondientes del stock.

3.4. Funci3n auxiliar: fn_calcular_descuento

Esta funci3n recibe la cantidad de tickets registrados previamente y devuelve un porcentaje de descuento dependiendo del rango en el que se encuentre el n3mero de tickets. Para ello utilizamos estructuras IF y ELSIF.

3.5. Trigger: trg_ticket_set_descuento

Este trigger se ejecuta antes de insertar un ticket. Primero asigna la fecha actual si no existe una fecha y luego cuenta los tickets registrados en el mismo a3o para calcular autom3ticamente el descuento usando la funci3n fn_calcular_descuento. Finalmente inicializa el precio bruto y neto en cero si vienen como NULL.

15	
16	-- 3) Cuando un cliente compra, el stock baja y el ticket recalcula precios:
17	INSERT INTO TICKET (IDTicket, IDSucursal, IDCliente, Fecha, Hora)
18	VALUES (1004, 1, 5, DATE '2026-05-24', TIME '10:00');
19	
20	INSERT INTO COMPRAR (IDTicket, IDMedicamento, Cantidad)
21	VALUES (1004, 10, 3);
22	
23	SELECT IDTicket, Fecha, PrecioBruto, DescuentoAplicado, PrecioNeto
24	FROM TICKET
25	WHERE IDTicket = 1004;
26	

idticket [PK] integer	fecha date	preciobruto numeric (12,2)	descuentoaplicado numeric (5,2)	precioneto numeric (12,2)
1	1004	2026-05-...	525.57	5.00
				499.29

3.6. Trigger: trg_ticket_recalcular_precios

Este trigger recalcula autom3ticamente el precio bruto y neto de un ticket cuando se inserta, actualiza o elimina una compra. Para ello suma el costo total de los medicamentos asociados al ticket y posteriormente aplica el descuento correspondiente para obtener el precio neto.

Data Output	Messages	Notifications		
SQL	Showing rows: 1 to 1			
	Page No: 1	of 1		
idticket [PK] integer	fecha date	preciobrut numeric (12,2)	descuentoap numeric (5,2)	precioneto numeric (12,2)
1	1004 2026-05-...	525.57	5.00	499.29